

architecture.md

****Production System Architecture****

****Status****: Production system preparing for full public go-live

****Authority****: Single human (CEO / Engineering Lead / CTO)

****Last Updated****: Current system state

****Context****:

- VISION.md defines purpose and boundaries
- BUSINESS_LOGIC.md defines workflows, authority, and irreversible actions
- DOMAIN_MODEL.md defines entities, ownership, states, and transitions
- This document describes structure and boundaries only (no code, no workflows, no business logic)

1. High-Level System Overview

Major Components

Component	Responsibility	Trust Boundary
Frontend (Next.js)	User interface, client-side rendering, user input collection	Untrusted (client-side)
Backend (Convex)	Business logic, data storage, authorization, transaction processing	Trusted (server-side)
Deployment Infrastructure	Hosting, routing, environment configuration	External dependency

Component Responsibilities

****Frontend (Next.js App Router)****:

- Renders user interface based on user role
- Collects user input and sends to backend
- Displays data received from backend
- Handles client-side routing and navigation
- Supports PWA installability and offline-capable web behavior via ``next-pwa``
- Can be packaged for Android using Capacitor as a mobile wrapper
- ****Does NOT**** enforce business rules or authorization
- ****Does NOT**** store sensitive data
- ****Does NOT**** make autonomous decisions

****Backend (Convex)****:

- Stores all entities (User, Listing, ListingUnit, WalletLedger, TraderInventory, BuyerPurchase, PurchaseWindow, StorageFeeDeduction, AdminAction, SystemSettings, Notification, RateLimitHit)
- Enforces business rules and authorization
- Validates all user actions server-side
- Generates UTIDs for all meaningful actions
- Processes atomic transactions (pay-to-lock, reversals)
- Enforces exposure limits and rate limits
- Logs admin actions and rate limit violations
- ****Does NOT**** trust client-side data
- ****Does NOT**** make autonomous decisions without human authorization

****Deployment Infrastructure**:**

- ****Vercel****: Hosts frontend, routes requests, manages environment variables
- ****Convex****: Hosts backend, manages database, executes functions, provides real-time subscriptions
- ****Capacitor Android wrapper****: Packages the same Next.js web build into an Android app shell; runtime still depends on the web build and Convex backend
- ****BLOCKED****: Backup and restore procedures are UNKNOWN (Convex managed, but operator access is UNKNOWN)

2. Component Breakdown

Frontend Component

****Purpose****: Provide user interface for all user roles (farmer, trader, buyer, admin)

****Owned Entities****: None (frontend does not own entities, only displays them)

****Read Permissions****:

- Reads all entities via Convex queries (subject to backend authorization)
- Cannot read entities directly (all reads go through backend)

****Write Permissions****:

- Cannot write entities directly (all writes go through backend mutations)
- Sends user input to backend for processing

****Trust Boundary Classification****: ****Untrusted****

- All user input must be validated server-side
- All authorization must be enforced server-side
- Frontend cannot bypass backend validation

Backend Component (Convex)

****Purpose****: Enforce business logic, store data, process transactions, authorize actions

****Owned Entities****: All entities from DOMAIN_MODEL.md:

- User
- Listing
- ListingUnit
- WalletLedger
- TraderInventory
- BuyerPurchase
- PurchaseWindow
- StorageFeeDeduction
- AdminAction
- SystemSettings
- Notification
- RateLimitHit

****Read Permissions****:

- Backend can read all entities (for authorization and business logic)
- Users can read entities only through authorized queries (enforced server-side)

****Write Permissions****:

- Backend can write all entities (subject to authorization rules)
- Users can write entities only through authorized mutations (enforced server-side)
- Admin can write: ListingUnit (delivery status), TraderInventory (status), PurchaseWindow (open/close), SystemSettings (pilot mode), User (role, suspend/unsuspend)
- System can write: WalletLedger, ListingUnit (via listing creation), TraderInventory (via delivery), StorageFeeDeduction, Notification, RateLimitHit
- ****BLOCKED****: StorageFeeDeduction automation write authority is UNKNOWN
- ****BLOCKED****: Delivery verification write authority is UNKNOWN

****Trust Boundary Classification****: ****Trusted****

- All business logic enforcement happens here
- All authorization enforcement happens here
- All transaction atomicity is guaranteed here

Deployment Infrastructure Component

****Purpose****: Host and route requests, manage environment configuration

****Owned Entities****: None (infrastructure does not own domain entities)

****Read Permissions**:**

- Infrastructure can read environment variables
- Infrastructure cannot read domain entities

****Write Permissions**:**

- Infrastructure cannot write domain entities
- Infrastructure can manage deployment configuration

****Trust Boundary Classification**:** ****External Dependency****

- Infrastructure is outside system control
- Infrastructure failures affect system availability
- Infrastructure does not enforce business logic

3. Data Layer Architecture

Where Data Lives

****All data lives in Convex database**:**

- All entities from DOMAIN_MODEL.md are stored in Convex tables
- No data is stored in frontend (client-side storage is not used for domain entities)
- No data is stored in external systems (no third-party business integrations)

****Data Storage by Entity**:**

Entity	Storage Location	Access Method
-----	-----	-----
User	Convex `users` table	Convex queries/mutations
Listing	Convex `listings` table	Convex queries/mutations
ListingUnit	Convex `listingUnits` table	Convex queries/mutations
WalletLedger	Convex `walletLedger` table	Convex queries/mutations
TraderInventory	Convex `traderInventory` table	Convex queries/mutations
BuyerPurchase	Convex `buyerPurchases` table	Convex queries/mutations
PurchaseWindow	Convex `purchaseWindows` table	Convex queries/mutations
StorageFeeDeduction	Convex `storageFeeDeductions` table	Convex queries/mutations
AdminAction	Convex `adminActions` table	Convex queries/mutations
SystemSettings	Convex `systemSettings` table	Convex queries/mutations
Notification	Convex `notifications` table	Convex queries/mutations
RateLimitHit	Convex `rateLimitHits` table	Convex queries/mutations

Immutability Guarantees

****Immutable Entities**** (entries are created and never modified):

- ****WalletLedger****: Entries are immutable. No updates or deletes. New entries are created for all changes.
- ****AdminAction****: Log entries are immutable. No updates or deletes.
- ****StorageFeeDeduction****: Deduction records are immutable. No updates or deletes.
- ****RateLimitHit****: Violation records are immutable. No updates or deletes.

****Mutable Entities**** (state transitions allowed, but history preserved):

- ****User****: State transitions (active → suspended → deleted) are allowed. Role changes are logged.
- ****Listing****: State transitions (active → partially_locked → fully_locked → delivered/cancelled) are allowed.
- ****ListingUnit****: State transitions (available → locked → delivered/cancelled) are allowed. Delivery status changes are logged.
- ****TraderInventory****: State transitions (pending_delivery → in_storage → sold/expired) are allowed.
- ****BuyerPurchase****: State transitions (pending_pickup → picked_up/expired) are allowed. ****BLOCKED****: Purchase function NOT IMPLEMENTED.
- ****PurchaseWindow****: State transitions (closed ↔ open) are allowed. Changes are logged.
- ****SystemSettings****: Setting changes (pilotMode: true ↔ false) are allowed. Changes are logged.
- ****Notification****: State transitions (unread → read) are allowed.

****UTID Immutability****:

- UTIDs are immutable once generated
- UTIDs cannot be modified or deleted
- UTIDs are used for audit trail and traceability

Backup and Restore Posture

****Backup****:

- ****BLOCKED****: Backup procedures are UNKNOWN
- Convex provides managed backups, but operator access and restore procedures are UNKNOWN
- No explicit backup verification exists

****Restore****:

- ****BLOCKED****: Restore procedures are UNKNOWN
- No explicit restore testing has been performed
- No explicit restore authorization process exists

****Data Loss Risk****:

- ****Risk Owner****: System operator

- **Mitigation**: None (backup/restore procedures are UNKNOWN)
- **BLOCKED**: Data loss recovery procedures cannot be specified until backup/restore procedures are verified

BLOCKED Areas

BLOCKED 1: Backup and Restore Procedures

- **Blocked By**: Backup and restore procedures are UNKNOWN
- **Impact**: Data loss recovery cannot be specified
- **What Would Unblock**: Verification of Convex backup/restore procedures and operator access

BLOCKED 2: Storage Fee Automation Data Access

- **Blocked By**: Storage fee automation implementation status is UNKNOWN
- **Impact**: StorageFeeDeduction entity write authority is UNKNOWN
- **What Would Unblock**: Verification and implementation of storage fee automation

BLOCKED 3: Delivery Verification Data Access

- **Blocked By**: Delivery verification function implementation status is UNKNOWN
- **Impact**: ListingUnit delivery status write authority is UNKNOWN
- **What Would Unblock**: Verification and implementation of delivery verification function

4. Authentication & Authorization Architecture

Current State

Authentication:

- Authentication module implementation exists in ``convex/authentication`` and supports stateful sessions, session validation, logout, and password reset flows
- Current pilot environment still uses shared pilot access controls; secure production authentication activation is pending
- **Role Assignment**: Authentication does not infer roles from email prefixes; explicit user role assignment is required from User Management
- **BLOCKED FOR PRODUCTION**: Production authentication activation, secure password hashing integration, and password reset email delivery are not yet fully enabled

Authorization:

- **Server-Side Enforcement**: All authorization is enforced in Convex backend
- **Role-Based Access**: Access is controlled by user role (farmer, trader, buyer, admin)

- ****Admin Authority****: Admin can verify deliveries, reverse transactions, control purchase window, enable/disable pilot mode, change user roles
- ****System Operator Authority****: System operator can activate/deactivate system, shutdown system
- ****BLOCKED****: Delivery verification authorization is UNKNOWN (function status is UNKNOWN)

Production Target State

****Authentication****:

- ****BLOCKED****: Production authentication target state cannot be specified (implementation is BLOCKED)
- ****Required****: Explicit role assignment (not inferred from email prefix)
- ****Required****: Production-grade authentication mechanism
- ****BLOCKED****: Target state depends on production authentication implementation

****Authorization****:

- ****Target****: Same as current state (server-side enforcement, role-based access)
- ****Target****: Explicit admin-controlled role assignment (not system-inferred)
- ****BLOCKED****: Authorization target state depends on production authentication implementation

Explicit BLOCKED Items

****BLOCKED 1: Production Authentication****

- ****Blocked By****: VISION.md BLOCKED item #1 (Production Authentication NOT IMPLEMENTED)
- ****Impact****: Production authentication architecture cannot be specified
- ****What Would Unblock****: Implementation of production authentication mechanism

****BLOCKED 2: Role Assignment Mechanism****

- ****Blocked By****: Role inference from email prefix is BLOCKED FOR PRODUCTION
- ****Impact****: Role assignment architecture for production cannot be specified
- ****What Would Unblock****: Implementation of explicit admin-controlled role assignment

****BLOCKED 3: Delivery Verification Authorization****

- ****Blocked By****: Delivery verification function implementation status is UNKNOWN
- ****Impact****: Delivery verification authorization architecture is UNKNOWN
- ****What Would Unblock****: Verification and implementation of delivery verification function

5. External Dependencies

Infrastructure Providers

Provider	Service	Failure Impact	Dependency Risk Ownership
-----	-----	-----	-----
Vercel	Frontend hosting, routing, environment variables	System unavailable (users cannot access frontend)	System operator
Convex	Backend hosting, database, function execution, real-time subscriptions	System unavailable (users cannot access backend, data cannot be read/written)	System operator

Failure Impact Analysis

****Vercel Failure**:**

- ****Impact****: Frontend is unavailable. Users cannot access system.
- ****Mitigation****: None (system depends on Vercel availability)
- ****Recovery****: Vercel must restore service. System operator cannot restore independently.
- ****Data Impact****: None (data is stored in Convex, not Vercel)

****Convex Failure**:**

- ****Impact****: Backend is unavailable. Users cannot access data or perform actions.
- ****Mitigation****: None (system depends on Convex availability)
- ****Recovery****: Convex must restore service. System operator cannot restore independently.
- ****Data Impact****: ****BLOCKED****: Data recovery procedures are UNKNOWN (backup/restore procedures are UNKNOWN)

Dependency Risk Ownership

****System Operator Bears Risk For**:**

- Vercel availability (frontend hosting)
- Convex availability (backend hosting, database)
- Convex data integrity (database integrity guarantees)
- ****BLOCKED****: Convex backup/restore access (procedures are UNKNOWN)

****No Third-Party Business Integration Risks**:**

- System does not integrate with external business services (banks, payment processors, SMS providers)
- No third-party business integration risks exist (by design)

6. Kill-Switch & Shutdown Design

What Can Be Stopped

****Pilot Mode (Admin-Controlled Kill-Switch)**:**

- ****What It Stops**:** All money-moving mutations (capital deposits, capital locks, profit withdrawals, unit locks)
- ****Who Can Stop It**:** Admin only
- ****How It Works**:** Admin sets ``systemSettings.pilotMode = true``. Backend blocks all mutations that move money or inventory.
- ****Reversibility**:** Reversible (admin can set ``pilotMode = false`` to re-enable mutations)
- ****What Continues**:** Read-only operations continue (queries, data display)
- ****What Must Never Continue**:** Money-moving mutations must never continue when pilot mode is enabled

****Purchase Window (Admin-Controlled Kill-Switch)**:**

- ****What It Stops**:** Buyer purchases (buyers cannot purchase inventory)
- ****Who Can Stop It**:** Admin only
- ****How It Works**:** Admin sets ``purchaseWindows.isOpen = false``. Backend blocks buyer purchase mutations.
- ****Reversibility**:** Reversible (admin can set ``isOpen = true`` to re-enable purchases)
- ****What Continues**:** All other operations continue (farmers can list, traders can lock units, etc.)
- ****What Must Never Continue**:** Buyer purchases must never continue when purchase window is closed

****System Shutdown (System Operator-Controlled Kill-Switch)**:**

- ****What It Stops**:** Entire system (frontend and backend become unavailable)
- ****Who Can Stop It**:** System operator (CEO / Engineering Lead / CTO) only
- ****How It Works**:** System operator shuts down Vercel deployment and/or Convex deployment
- ****Reversibility**:** Reversible (system operator can restart deployments)
- ****What Continues**:** Nothing (system is completely unavailable)
- ****What Must Never Continue**:** No operations must continue after system shutdown

Who Can Stop It

Kill-Switch	Authority	Enforcement	Auditability
-----	-----	-----	-----
Pilot Mode	Admin	Backend mutation blocking	AdminAction log entry with UTID and reason
Purchase Window	Admin	Backend mutation blocking	AdminAction log entry with UTID and reason

| System Shutdown | System Operator | Infrastructure shutdown | ****BLOCKED****:
Shutdown logging is UNKNOWN |

What Continues Running After Shutdown

****After Pilot Mode Activation****:

- Read-only queries continue
- Data display continues
- User authentication continues
- ****Must Never Continue****: Money-moving mutations

****After Purchase Window Closure****:

- All operations except buyer purchases continue
- Farmers can create listings
- Traders can lock units
- Admin can verify deliveries
- ****Must Never Continue****: Buyer purchases

****After System Shutdown****:

- Nothing continues (system is completely unavailable)
- ****Must Never Continue****: No operations

What Must Never Continue Running

****After Pilot Mode Activation****:

- Capital deposits
- Capital locks
- Profit withdrawals
- Unit locks (pay-to-lock)
- ****Enforcement****: Backend must block all mutations that move money or inventory

****After Purchase Window Closure****:

- Buyer purchases
- ****Enforcement****: Backend must block buyer purchase mutations

****After System Shutdown****:

- All operations
- ****Enforcement****: Infrastructure shutdown prevents all operations

****BLOCKED****: Shutdown logging and auditability for system shutdown are UNKNOWN.

7. Single-Human Authority Model

Where Authority is Exercised

****System Operator Authority**** (CEO / Engineering Lead / CTO):

- ****Production Activation****: System operator can activate system for public go-live
- ****System Shutdown****: System operator can shutdown system (emergency or planned)
- ****BLOCKED****: Production activation is NOT verified (see ``docs/architecture.md``)

****Admin Authority**** (Single human with admin role):

- ****Delivery Verification****: Admin can mark delivery as ``delivered``, ``late``, or ``cancelled``
- ****Transaction Reversal****: Admin can reverse transactions (unlock unit + unlock capital)
- ****Purchase Window Control****: Admin can open or close purchase window
- ****Pilot Mode Control****: Admin can enable or disable pilot mode
- ****User Role Changes****: Admin can change user roles
- ****BLOCKED****: Delivery verification function implementation status is UNKNOWN

How Authority is Enforced

****Server-Side Enforcement****:

- All authority checks are enforced in Convex backend
- Frontend cannot bypass authority checks
- All mutations require server-side authorization

****Role-Based Enforcement****:

- Admin actions require ``role === "admin"`` (verified server-side)
- System operator actions require infrastructure access (outside application code)
- Users cannot change their own role
- Users cannot bypass exposure limits

****Pilot Mode Enforcement****:

- When ``systemSettings.pilotMode === true``, backend blocks all money-moving mutations
- Enforcement is server-side (frontend cannot bypass)
- ****BLOCKED****: Pilot mode enforcement implementation status is UNKNOWN (assumed to exist)

How Authority is Audited

****Admin Actions****:

- All admin actions are logged in ``adminActions`` table
- Each log entry includes: adminId, actionType, utid, reason, targetUtid, timestamp

- Log entries are immutable (cannot be modified or deleted)
- ****BLOCKED****: Delivery verification actions may not be logged if function is not implemented

****System Operator Actions****:

- ****BLOCKED****: System operator actions (production activation, system shutdown) are not logged in system
- System operator actions are outside application code (infrastructure level)
- ****BLOCKED****: Shutdown logging and auditability are UNKNOWN

****User Actions****:

- User actions are tracked via UTIDs (all meaningful actions generate UTIDs)
- UTIDs are immutable and cannot be modified
- UTIDs provide audit trail for all transactions

****BLOCKED****: Complete auditability for system operator actions is UNKNOWN.

8. BLOCKED SUMMARY

Architectural Elements Blocked by Unresolved Items

Architectural Element	Blocked Item	Why Blocked	What Would Unblock
----- ----- ----- -----			
Production Authentication Architecture	VISION.md BLOCKED #1	Production authentication NOT IMPLEMENTED	Implementation of production authentication mechanism
Role Assignment Architecture	Role inference BLOCKED FOR PRODUCTION	Role inference from email prefix is BLOCKED FOR PRODUCTION	Implementation of explicit admin-controlled role assignment
Delivery Verification Architecture	VISION.md UNKNOWN	Delivery verification function implementation status UNKNOWN	Verification and implementation of delivery verification function
Storage Fee Automation Architecture	VISION.md UNKNOWN	Storage fee automation implementation status UNKNOWN	Verification and implementation of storage fee automation
Backup and Restore Architecture	Backup/restore procedures UNKNOWN	Backup and restore procedures are UNKNOWN	Verification of Convex backup/restore procedures and operator access
System Shutdown Auditability	Shutdown logging UNKNOWN	Shutdown logging and auditability are UNKNOWN	Implementation of system shutdown logging and auditability

| ****Pilot Mode Enforcement**** | Enforcement implementation UNKNOWN | Pilot mode enforcement implementation status is UNKNOWN (assumed to exist) | Verification of pilot mode enforcement implementation |

| ****Buyer Purchase Architecture**** | VISION.md BLOCKED #2 | Buyer purchase function NOT IMPLEMENTED | Implementation of buyer purchase function |

Trust Boundaries Blocked by Unresolved Items

Trust Boundary	Blocked Item	Why Blocked	What Would Unblock
-----	-----	-----	-----

| ****Production Authentication Trust Boundary**** | VISION.md BLOCKED #1 | Production authentication NOT IMPLEMENTED | Implementation of production authentication mechanism |

| ****Delivery Verification Trust Boundary**** | VISION.md UNKNOWN | Delivery verification function implementation status UNKNOWN | Verification and implementation of delivery verification function |

| ****Storage Fee Automation Trust Boundary**** | VISION.md UNKNOWN | Storage fee automation implementation status UNKNOWN | Verification and implementation of storage fee automation |

Kill-Switch Points Blocked by Unresolved Items

Kill-Switch	Blocked Item	Why Blocked	What Would Unblock
-----	-----	-----	-----

| ****Pilot Mode Enforcement**** | Enforcement implementation UNKNOWN | Pilot mode enforcement implementation status is UNKNOWN | Verification of pilot mode enforcement implementation |

| ****System Shutdown Auditability**** | Shutdown logging UNKNOWN | Shutdown logging and auditability are UNKNOWN | Implementation of system shutdown logging and auditability |

Final Check

All Components Defined

****Verified****: All components are defined:

1. Frontend (Next.js)
2. Backend (Convex)
3. Deployment Infrastructure (Vercel, Convex)

All Trust Boundaries Identified

****Verified****: All trust boundaries are identified:

1. Frontend: Untrusted (client-side)
2. Backend: Trusted (server-side)
3. Deployment Infrastructure: External Dependency

All Kill-Switch Points Listed

****Verified****: All kill-switch points are listed:

1. Pilot Mode (Admin-controlled)
2. Purchase Window (Admin-controlled)
3. System Shutdown (System Operator-controlled)

All BLOCKED Architectural Risks Explicitly Marked

****Verified****: All BLOCKED architectural risks are explicitly marked:

1. Production Authentication Architecture: BLOCKED
2. Role Assignment Architecture: BLOCKED FOR PRODUCTION
3. Delivery Verification Architecture: BLOCKED
4. Storage Fee Automation Architecture: BLOCKED
5. Backup and Restore Architecture: BLOCKED
6. System Shutdown Auditability: BLOCKED
7. Pilot Mode Enforcement: BLOCKED (assumed to exist)
8. Buyer Purchase Architecture: BLOCKED

Confirmation: No New Authority or Entities Introduced

****Verified****: No new authority or entities were introduced:

- All entities are from DOMAIN_MODEL.md (12 entities)
- All authority is from BUSINESS_LOGIC.md (Admin authority, System Operator authority)
- All roles are from DOMAIN_MODEL.md (farmer, trader, buyer, admin)
- No new components, entities, roles, or authority were introduced

This document must be updated when architecture changes, BLOCKED items are unblocked, or new components are introduced. No assumptions. Only truth.